

NVIDIA Sybil — Reference Architecture (as the project documents it)

sybild runs ALONGSIDE the KDC and impersonates users via S4U for Slurm batch jobs • MIT (LDAP backend) or FreeIPA / RHEL IdM
one Linux Kerberos realm — NOT Active Directory (AD is not in Sybil's supported environment: for the AD client adaptation see krb-credd-architecture)

SINGLE KERBEROS REALM — MIT Kerberos (LDAP backend) or FreeIPA / RHEL IdM • all Linux

KDC TRUST CORE — co-located “alongside the KDC”

MIT KDC (LDAP backend) / FreeIPA IdM

- AS / TGS (Kerberos, TCP 88)

LDAP directory (backend — required for S4U)

- krbAllowedToDelegateTo = { svc/... }
 ← the S4U2Proxy allow-list
- target services flagged +ok_as_delegate
- “Allow delegation to the Sybil server”

↕ S4U2Self + S4U2Proxy
(authorized by LDAP)

sybild — privileged delegation daemon

- hosted ALONGSIDE the KDC
- impersonates users via S4U2Self + S4U2Proxy (GSSAPI)
- ACLs in /etc/sybil.toml • sybil service keytab
- NOT a KDC — no master key; uses a service principal

↓ store delegated creds

KCM — Kerberos Credential Manager

- “store delegated credentials” via the KCM protocol
- deployed alongside the KDC + sybild
- the credential store the job's creds come from

HPC / SLURM

Login / submit node

- authorized principal: kinit -k
- sybil kinit user@DOMAIN.LAN (CLI → sybild)
- authenticated delegation — not blanket unattended

↓ submit

Slurm controller

- sbatch --kerberos=yes
- SPANK plugin hooks the job lifecycle

↓ launch tasks

Compute nodes cn001..cnNNN

- SPANK plugin: forward + renew the user's creds across the job lifecycle (pulled from KCM)
- job authenticates as the user

↓ use creds (GSSAPI)

Kerberized backends (same realm)

- NFS / Lustre / HDFS / ...
- accept the delegated service tickets over GSSAPI

kinit -k (user)

sybil kinit
user@DOMAIN

SPANK: forward +
renew creds
(--kerberos=yes)